

Angular CLI

Comando	Descripción
ng new app	Crea un proyecto nuevo
ng serve	Sirve en localhost:4200
ng generate component x	Genera un componente (g c x)
ng g service x	Genera un servicio
ng g guard x	Genera un guard
ng build	Build de producción a dist/
ng test	Corre pruebas unitarias
ng add @angular/material	Instala y configura Material

Componente standalone

El bloque base de la UI. Desde v17 es el default; en v18 se declara `standalone: true` (implícito en v19+).

```
@Component({
  selector: 'app-libro',
  standalone: true,
  imports: [CommonModule],
  templateUrl: './libro.component.html'
})
export class LibroComponent {
  titulo = 'Clean Code';
}
```

Arranque (main.ts / app.config)

```
bootstrapApplication(AppComponent, appConfig);

export const appConfig = {
  providers: [
    provideRouter(routes),
    provideHttpClient()
  ]
};
```

Signals

```
count = signal(0); // estado
doble = computed(() => this.count() * 2);
effect(() => console.log(this.count()));

this.count.set(5); // reemplaza
this.count.update(n => n + 1); // desde el actual
```

i Lee un signal llamándolo: `count()`. Actualiza con `set/update` de forma inmutable. Usa `computed` para derivar, no `effect`.

Control flow (v17+)

```
@if (libros().length) {
  @for (l of libros(); track l.id) {
    <app-card [libro]="l" />
  }
} @else {
  <p>Sin datos</p>
}
@switch (estado()) {
  @case ('ok') { <ok/> }
  @default { <x/> }
}
```

Comunicación de componentes

```
// hijo (signals)
libro = input.required<Libro>();
sel = output<Libro>();
valor = model(0); // two-way
```

```
<!-- padre -->
<app-card [libro]="l"
  (sel)="ver($event)"
  [(valor)]="total" />
```

Inyección de dependencias

```
@Injectable({ providedIn: 'root' })
export class LibroService {
  private http = inject(HttpClient);
}
// en componente:
srv = inject(LibroService);
```

HttpClient (CRUD)

```
listar(): Observable<Libro[]> {
  return this.http.get<Libro[]>(url);
}
crear(dto) { return this.http.post<Libro>(url, dto); }
editar(id,d){ return this.http.put<Libro>(`${url}/${id}`,
  d); }
borrar(id) { return
  this.http.delete<void>(`${url}/${id}`); }
```

```
// consumir sin fugas
libros = toSignal(this.srv.listar(),
  { initialValue: [] });
// o en template: {{ data$ | async }}
```

Params, headers y errores

```
const params = new HttpParams()
  .set('page', 1).set('size', 10);
this.http.get(url, { params }).pipe(
  retry(2),
  catchError(e => of([]))
);
```

RxJS: operadores clave

Operador	Uso
map	Transforma cada valor
filter	Deja pasar los que cumplen
switchMap	Cancela la anterior (búsquedas)
mergeMap	En paralelo, sin orden
concatMap	En cola, respeta el orden
exhaustMap	Ignora nuevas (anti doble-click)
forkJoin	Espera a que todos completen
combineLatest	Últimos valores al cambiar
debounceTime	Espera a que pare de emitir
catchError	Maneja el error del flujo

Autocompletado reactivo

```
this.buscar.valueChanges.pipe(
  debounceTime(300),
  distinctUntilChanged(),
  switchMap(t => this.srv.buscar(t))
).subscribe(r => this.res.set(r));
```

Formularios reactivos

```
form = this.fb.group({
  titulo: ['', Validators.required],
  email: ['', [Validators.required,
    Validators.email]],
  precio: [0, Validators.min(1)]
});
guardar() {
  if (this.form.invalid) return;
  console.log(this.form.value);
}
```

```
<form [formGroup]="form"
  (ngSubmit)="guardar()">
  <input formControlName="titulo" />
  @if (form.get('titulo')?.touched &&
    form.get('titulo')?.invalid) {
    <small>Requerido</small>
  }
</form>
```

Validadores

Validator	Regla
required	Campo obligatorio
email	Formato de email
min / max	Rango numérico
minLength	Longitud mínima
pattern	Expresión regular

Router y lazy loading

```
export const routes: Routes = [
  { path: '', component: HomeComponent },
  {
    path: 'libros',
    canActivate: [AuthGuard],
    loadChildren: () =>
      import('./libros/list.component')
        .then(m => m.ListComponent)
  }
];
```

```
<a routerLink="/libros">Libros</a>
<router-outlet />
// leer params:
id = inject(ActivatedRoute).snapshot
  .params['id'];
```

Interceptor + guard

```
export const authInterceptor:
  HttpInterceptorFn = (req, next) => {
  const t = localStorage.getItem('token');
  if (t) req = req.clone({
    setHeaders: { Authorization: `Bearer ${t}` }
  });
  return next(req);
};
// provideHttpClient(withInterceptors([authInterceptor]))
```

```
export const authGuard: CanActivateFn =
  () => inject(Auth).logueado()
    ? true
    : inject(Router).createUrlTree(['/login']);
```

Angular Material

```
<mat-form-field appearance="outline">
  <mat-label>Titulo</mat-label>
  <input matInput formControlName="titulo">
  <mat-error>Requerido</mat-error>
</mat-form-field>
<button mat-raised-button color="primary">
  Guardar
</button>
```

Exportar Excel / PDF

```
// Excel (SheetJS)
const h = XLSX.utils.json_to_sheet(datos);
const wb = XLSX.utils.book_new();
XLSX.utils.book_append_sheet(wb, h, 'Datos');
XLSX.writeFile(wb, 'reporte.xlsx');
```

```
// PDF (jsPDF + autotable)
const doc = new jsPDF();
autoTable(doc, { head, body });
doc.save('reporte.pdf');
```

Pipes útiles

Pipe	Ejemplo
date	{{ f date:'dd/MM/yyyy' }}
currency	{{ p currency }}
uppercase	{{ t uppercase }}
async	{{ obs\$ async }}
json	{{ obj json }}

Buenas prácticas

- ✓ HTTP en servicios, no en componentes.
- ✓ Tipa todo con interfaces (evita **any**).
- ✓ **async** pipe / **toSignal** para no fugar suscripciones.
- ✓ Organiza por feature (core / shared / features).
- ! La seguridad real vive en el backend.